“Quando la passione incontra la tecnologia”

Management AI

Un agente AI ibrido per l’area sportiva e il management di un club di Serie A

Tesi finale del Master



Andrea Gervasio · 01

IL PROBLEMA

Le informazioni ci sono, ma non sono utilizzabili

Due aree, gli stessi giocatori: la direzione (direttore sportivo, segreteria, area legale e commerciale) e l'Area Performance & Medical (preparatori, staff medico, match analyst, allenatore).

Regole sparse nei documenti

Sanzioni, convocazioni, protocollo di return-to-play, soglie di carico e procedure sono in manuali PDF che nessuno ha il tempo di consultare durante la settimana.

Dati difficili da interrogare

Le statistiche delle partite (GPS, xG, wellness, infortuni) arrivano da più sistemi in un file "sporco", e ogni analisi richiede il tempo di un match analyst.

Contratti e scadenze sparsi

Contratti, accordi con gli sponsor, offerte di mercato e certificati medici sono file separati; per sapere quando scade un contratto o quanto vale un bonus bisogna aprirli uno per uno.

⚠ **Conseguenza:** le decisioni su carichi, rientri dagli infortuni e rotazioni si prendono senza incrociare regole e numeri.

LA SOLUZIONE IN UNA FRASE

“Un assistente che legge i documenti del club e analizza i dati delle partite, e decide da solo quale delle due cose fare per rispondere.”

RAG

Chiedi una regola

“Quali sono le fasi del return-to-play dopo un infortunio muscolare?”

Data agent

Chiedi un’analisi

“Chi sono i 10 migliori marcatori della stagione 2025-26?”

RAG Data agent

Chiedi entrambe le cose

“Quali giocatori hanno un wellness medio sotto la soglia di allerta del protocollo?”

L’agente prende la soglia dal protocollo e la applica ai dati.

VALORE PER IL REPARTO E PER IL CLUB

Risposte rapide, basate su regole e dati, sempre verificabili

Tempo

Risposte in secondi invece di cercare nei manuali o aspettare un report.

Regole + dati

Le soglie del club (ACWR > 1,5, wellness < 5, obiettivo < 20 infortuni ogni 1.000 ore) applicate ai numeri reali.

Fiducia e verifica

Fonti con link alla pagina del documento e codice Python eseguito. Niente numeri inventati dall'LLM.

Controllo dei costi

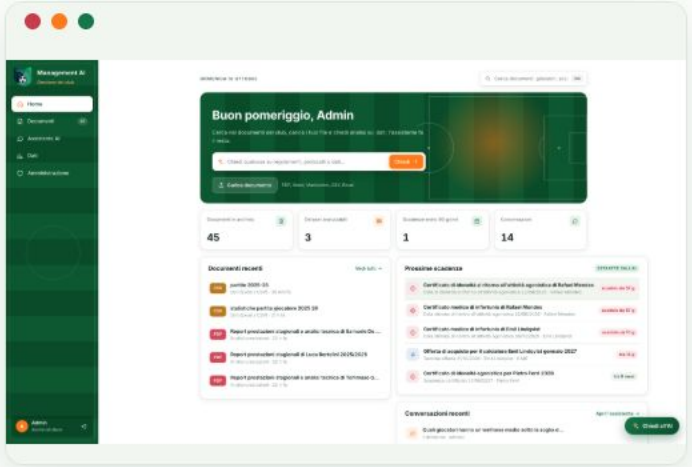
Dashboard admin con spesa AI per utente e per funzione, costo medio di circa 1-2 centesimi di dollaro per domanda e tetto di spesa giornaliero.

Una piattaforma, non solo una chat

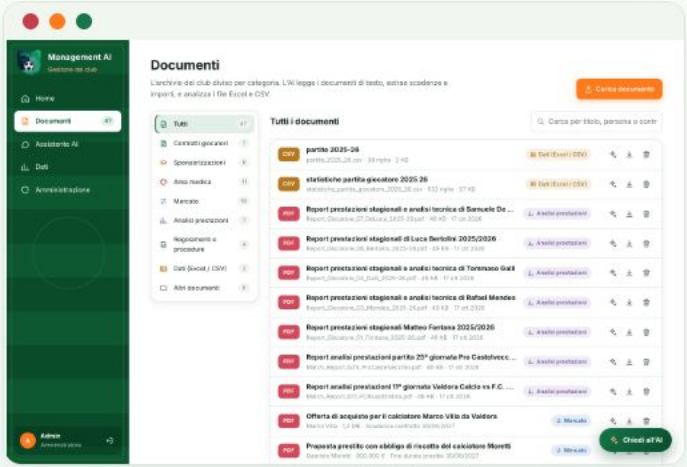
Archivio per categoria (contratti, sponsor, area medica, mercato), scadenze estratte in automatico, analisi di Excel/CSV caricati dallo staff.

UN GIRO NELLA PIATTAFORMA

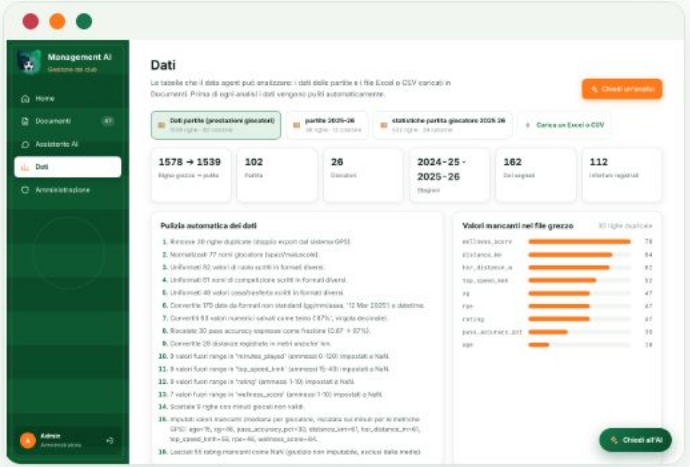
Tutto il reparto in un’unica piattaforma



Home · prossime scadenze



Documenti · archivio per categorie



Dati · pulizia automatica

L’assistente è disponibile in ogni pagina, dal pulsante **Chiedi all’AI**.



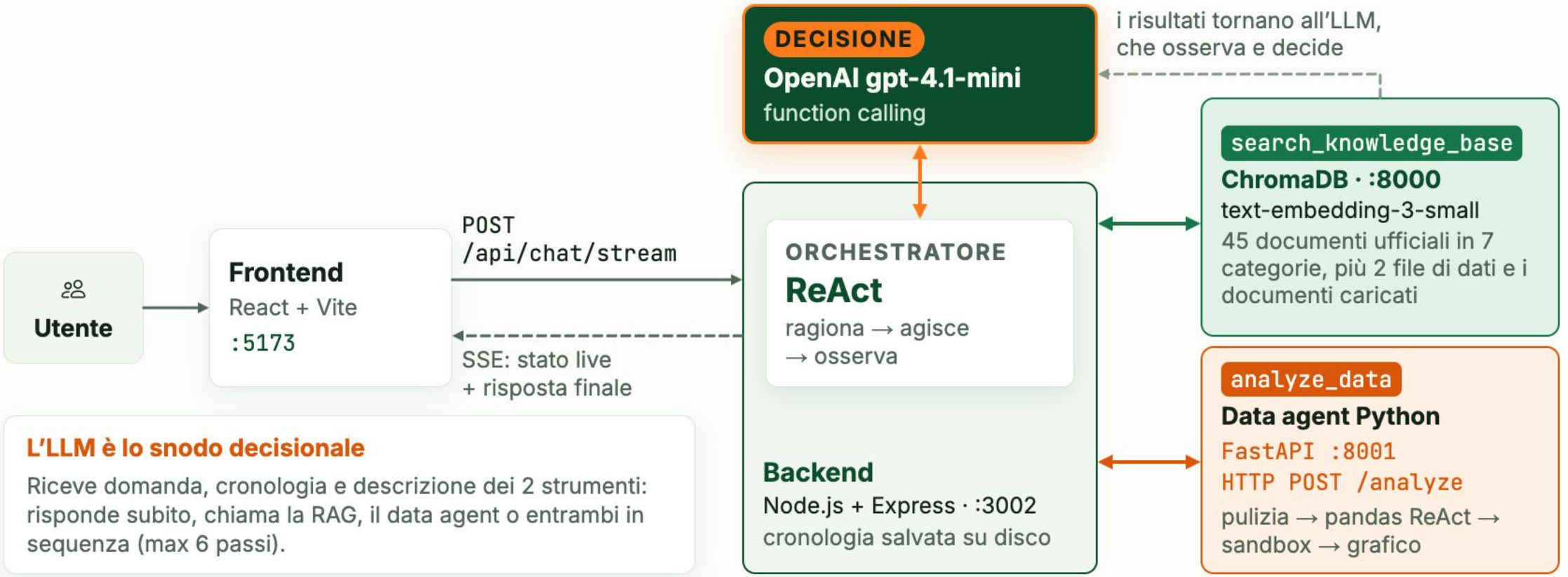
PARTE 2

Come funziona

Architettura, ciclo decisionale dell'agente, comunicazione, stack tecnologico e dati.

ARCHITETTURA DEL SISTEMA

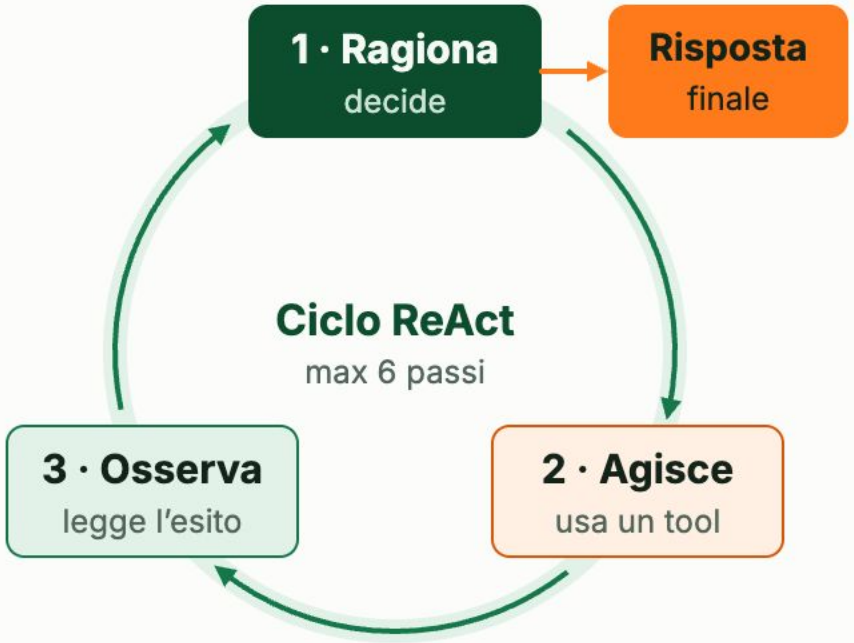
L'LLM decide, due strumenti eseguono



4 processi indipendenti, un solo file **.env**, avvio con **npm run dev**

COME DECIDE L'AGENTE: IL CICLO REACT

Ragiona, agisce, osserva, poi risponde

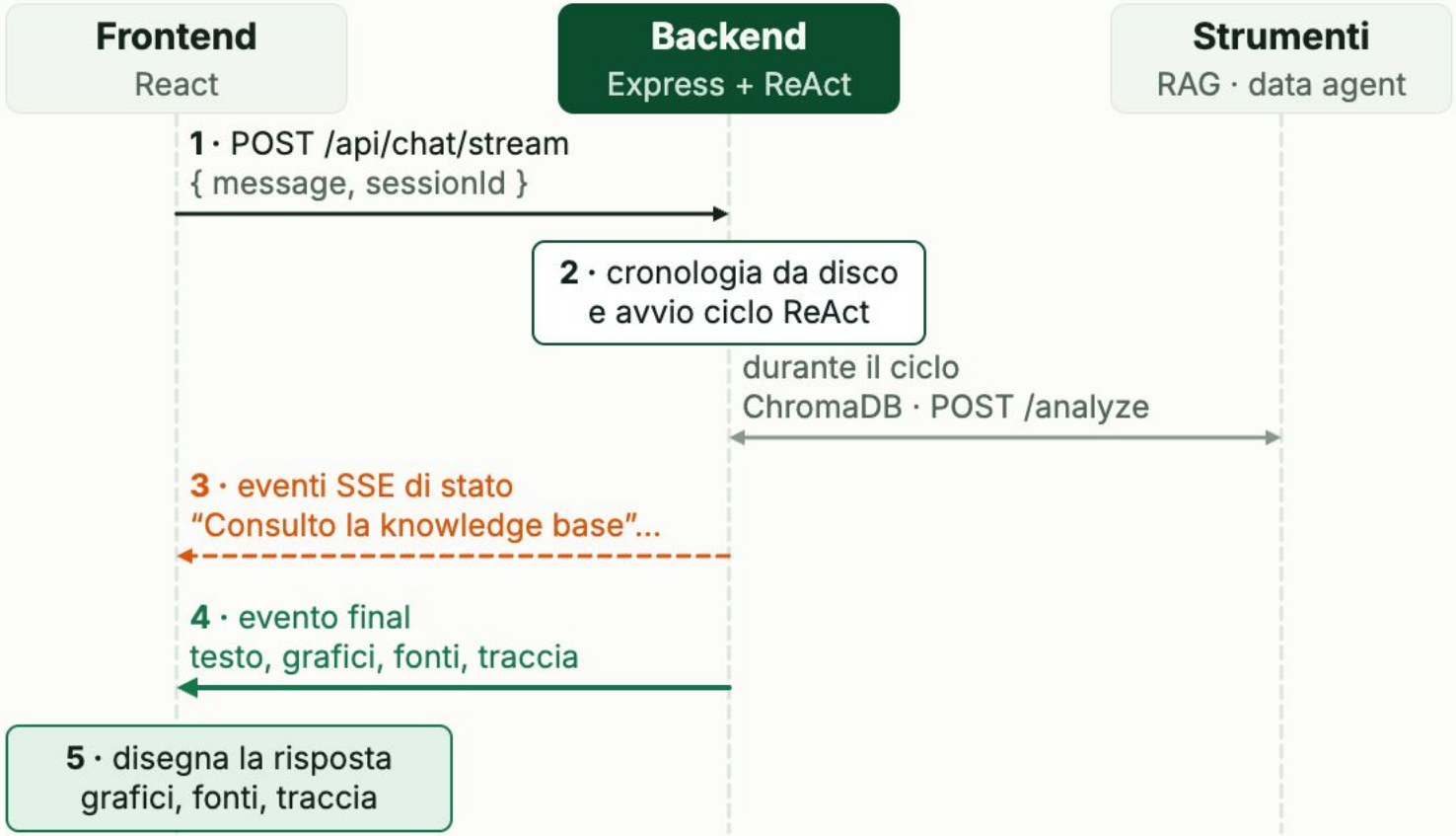


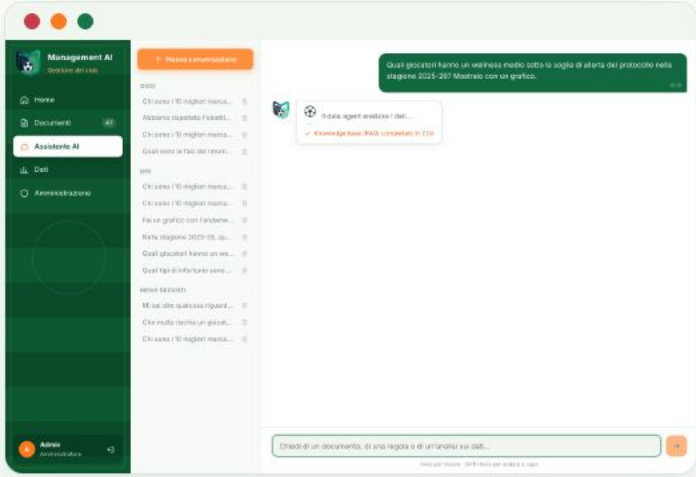
Domanda	Decisione dell'LLM
"Ciao, cosa sai fare?"	nessuno strumento
"Quali sono le fasi del return-to-play?"	search_knowledge_base (RAG)
"Chi sono i 10 migliori marcatori 2025-26?"	analyze_data (data agent)
"Chi ha un wellness sotto la soglia di allerta?"	prima RAG (trova la soglia), poi data agent (gli passa la soglia nel campo context)

‹/› Anche il data agent Python è un agente ReAct, con un solo strumento, **run_python**: scrive codice pandas, lo esegue davvero, legge l'output o l'errore e si corregge.

COME PARLANO FRONTEND E BACKEND

Una richiesta, eventi live, una risposta





Lo stato live arriva via SSE

Altri canali del backend

/analyze, /datasets → data agent

/api/charts/:file → proxy PNG

client chromadb → ChromaDB

Ogni scelta tecnica ha un motivo

Livello	Tecnologia	Perché questa scelta
Frontend	React 19, Vite, React Router, Recharts, react-markdown	Chat reattiva con stato condiviso tra pagina e pannello laterale; grafici interattivi nello stile dell'app; il codice dei grafici viene caricato solo quando serve
Comunicazione	Server-Sent Events	Stato live dell'agente senza la complessità dei WebSocket: il flusso va solo dal server al client
Backend	Node.js 20+, Express 5, architettura a livelli (routes / services / repositories / clients)	Orchestratore leggibile e testabile; i test usano l'app senza avviare il server
LLM	OpenAI SDK nativo, gpt-4.1-mini, function calling	Ciclo ReAct scritto in chiaro e tracciato passo per passo, senza LangChain: pieno controllo e niente incompatibilità di versione

STACK TECNOLOGICO E PERCHÉ · 2/2

In locale o online, sicuro e testato

Livello	Tecnologia	Perché questa scelta
RAG	ChromaDB locale, embedding text-embedding-3-small, distanza coseno	Database vettoriale che gira in locale senza installazioni separate; chunking per sezione e pagina, così le fonti citano la pagina giusta
Data agent	Python, FastAPI, pandas, numpy, matplotlib	Microservizio sempre attivo: pulisce il CSV una volta all'avvio, si testa da solo, documentazione automatica su /docs
Persistenza	File JSON con scrittura atomica, con lock per conversazione e file temporanei unici	Pochi dati locali: niente database da installare; ChromaDB resta il database dei documenti
Sicurezza	scrypt, cookie HttpOnly firmato HMAC, sanitize-html, sandbox AST	Login, protezione da XSS nei documenti Word, contenimento del codice generato dall'LLM
Qualità	node:test, pytest, npm audit, pip-audit, GitHub Actions, uv	Test automatici che non chiamano OpenAI e non toccano i dati reali; dipendenze senza vulnerabilità note. GitHub Actions a ogni push (test e audit); deploy solo con i test verdi; lock file delle dipendenze Python (uv)

I DATI E LA KNOWLEDGE BASE

Due fonti complementari

Dataset partite `player_performance.csv`

1.578 × 40
righe × colonne

102 partite
26 giocatori · 2024-25 e 2025-26

8 tipi
di anomalie introdotte apposta

1.578 → 1.539
righe dopo la pulizia

Dati GPS, xG, wellness, carico, infortuni. Anomalie: duplicati, valori mancanti, date in 3 formati, nomi incoerenti, alias di ruoli e competizioni, numeri salvati come testo, scale sbagliate, valori fuori range.



Knowledge base RAG

45 documenti ufficiali

- 4 regolamenti e procedure (regolamento, protocollo medico, linee guida, FAQ)
- 11 area medica
- 10 mercato
- 7 contratti
- 7 analisi prestazioni
- 6 sponsorizzazioni

+ 2 dataset Excel/CSV e i documenti caricati dallo staff (PDF, Word, Markdown, TXT)

I documenti definiscono le soglie, il dataset contiene i numeri a cui applicarle.

PARTE 3

L’agente al lavoro

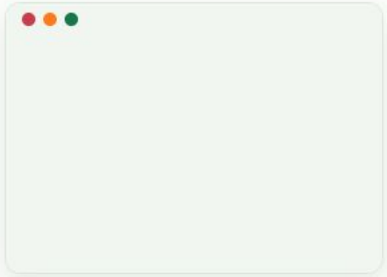
Tre domande reali: un documento, un’analisi dei dati e una domanda ibrida.

DEMO 1 · DOMANDA SULLA KNOWLEDGE BASE (RAG)

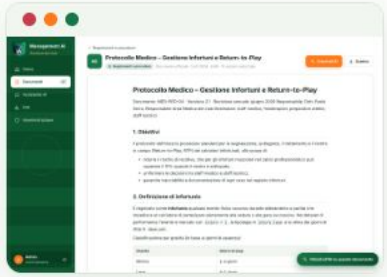
La risposta cita il documento e la sezione

> “Quali sono le fasi del return-to-play dopo un infortunio muscolare?”

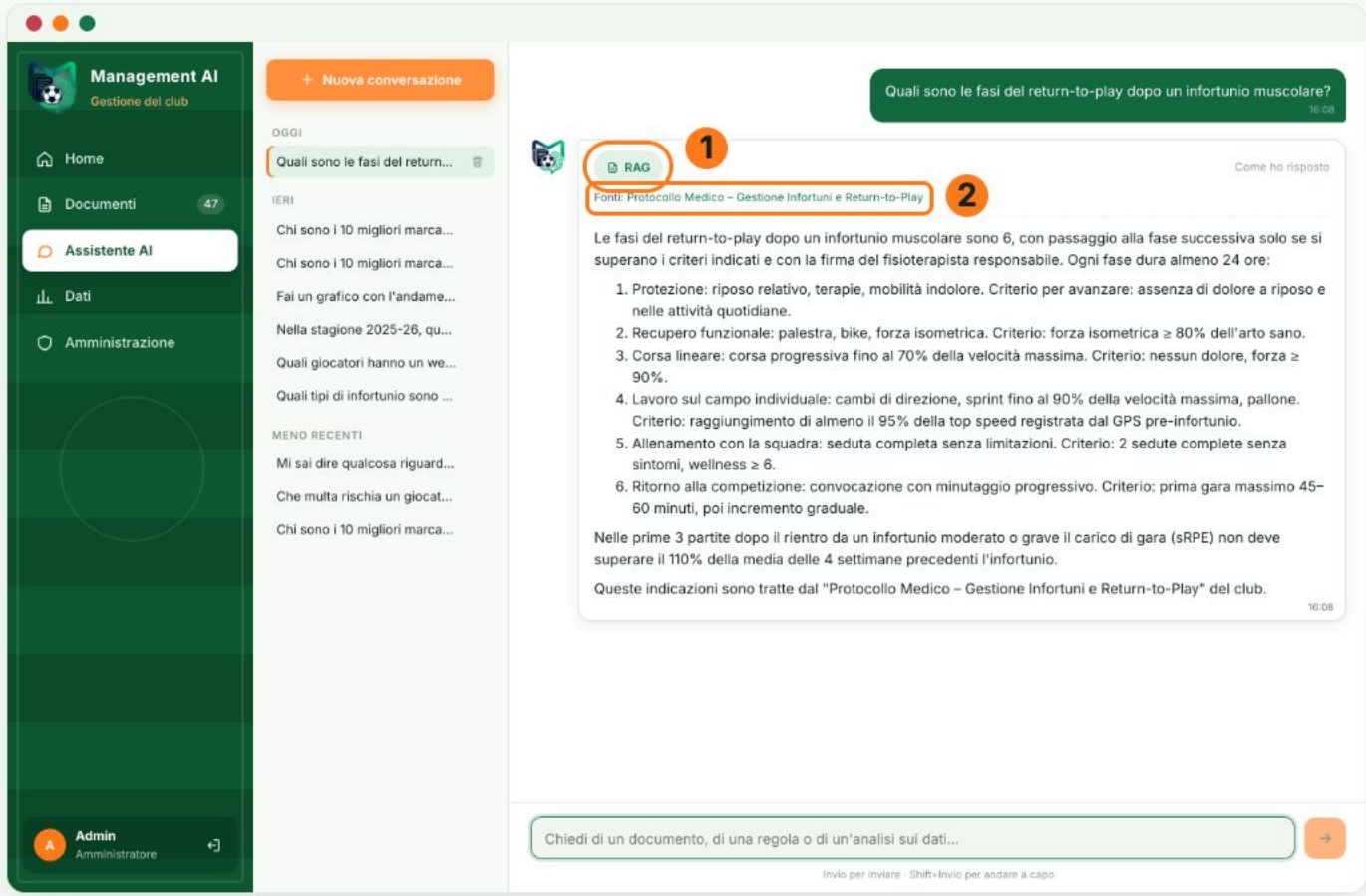
1. **Badge RAG**: l’agente ha scelto la knowledge base
2. **Fonti**: un clic apre il Protocollo Medico alla pagina giusta



“Come ho risposto”: chunk e similarità



Il protocollo aperto dalla fonte



DEMO 2 · ANALISI DEL DATASET CON GRAFICO (DATA AGENT)

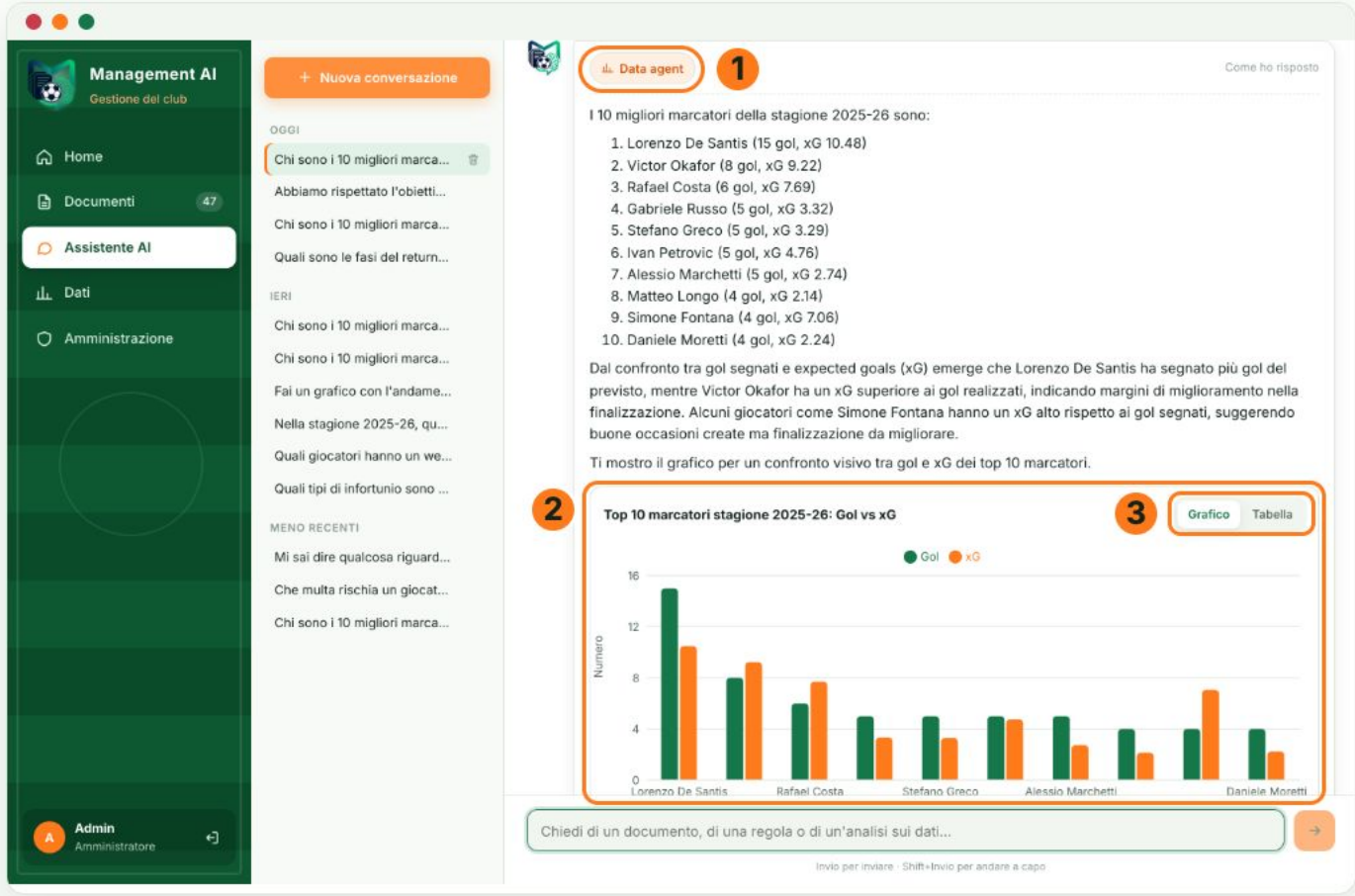
I numeri vengono da codice eseguito davvero

"Chi sono i 10 migliori marcatori della stagione 2025-26? Confronta gol ed xG con un grafico."

1. **Badge Data agent**: strumento usato
2. **Grafico gol vs xG**, generato in chat
3. **Grafico o tabella**, e "Scarica PNG"



"Come ho risposto": report di pulizia e codice pandas eseguito



DEMO 3 · DOMANDA IBRIDA (RAG + DATA AGENT)

Due strumenti, nell'ordine giusto

"Quali giocatori hanno un wellness medio sotto la soglia di allerta del protocollo nella stagione 2025-26? Mostralo con un grafico."

1 badge 2 fonti 3 grafico

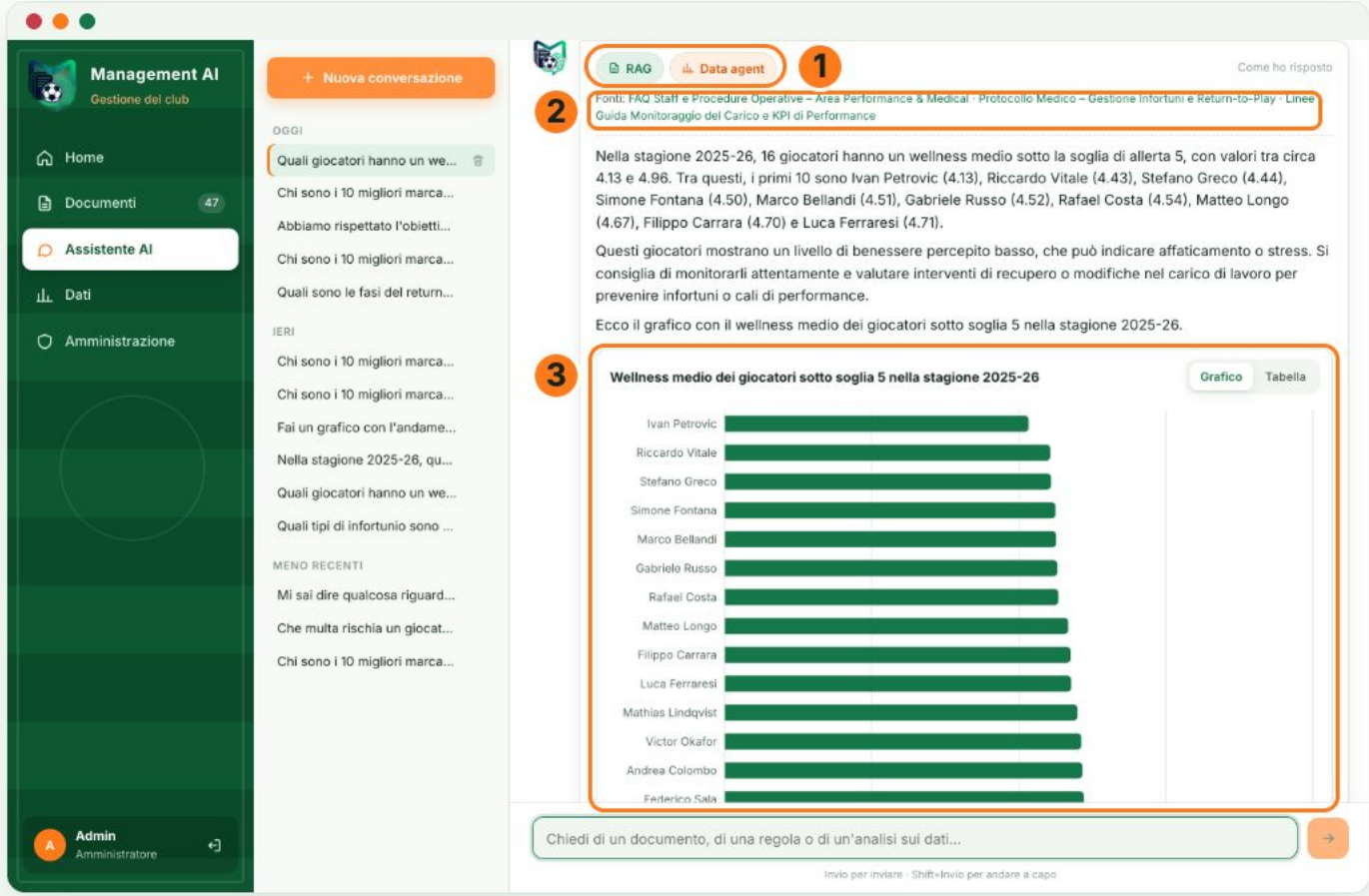
Passo 1 · RAG

Trova la soglia "wellness < 5" nelle Linee guida

la soglia passa dalla knowledge base all'analisi

Passo 2 · Data agent

La applica ai dati 2025-26 e disegna il grafico



Dati affidabili e un agente che si corregge

PROBLEMA	COME L'HO CAPITO	SOLUZIONE
1 **Dati sporchi da più sistemi**	Date in 3 formati, distanze in metri invece che in km, percentuali come frazioni, ruoli scritti in modi diversi: senza pulizia le medie sarebbero sbagliate senza dare nessun errore.	Pipeline di pulizia **deterministica** in codice (cleaning.py) prima dell'LLM, con report di ogni correzione e test sul risultato (1.578 → 1.539 righe). Il modello non "pulisce a occhio". **File caricati dallo staff:** conversioni prudenti, un valore ambiguo come 1.200 o 03/04/2025 resta testo, e il report dice perché.
2 **L'agente sbaglia il codice e si corregge da solo**	Al secondo tentativo il codice dà **ValueError**: l'agente legge l'errore, capisce che deve passare il nome di una colonna invece dell'indice e al terzo tentativo crea il grafico.	Gli errori tornano al modello come messaggi chiari e brevi, e le funzioni **chart()** e **table()** rispondono spiegando come correggere i dati.

Codice contenuto e grafici coerenti

PROBLEMA	COME L'HO CAPITO	SOLUZIONE
3 **Codice generato dall'LLM = codice non fidato**	Un documento caricato con istruzioni nascoste (prompt injection) poteva far scrivere al modello codice che leggeva il file .env con la chiave OpenAI.	Sandbox a **4 barriere**: analisi AST, builtins ridotti, I/O di pandas/numpy disattivato tramite proxy, timeout. Test che provano a leggere il .env, usare la rete e scrivere su disco.
4 **Domande ambigue sulla stagione**	Chiedendo "i migliori marcatori" senza stagione, l'agente a volte sceglieva il 2024-25 come "stagione attuale".	Il system prompt passa i dataset disponibili con le colonne, e nelle domande si indica la stagione. La traccia "Come ho risposto" mostra subito il filtro usato.
5 **Grafici fragili e incoerenti**	Con matplotlib libero, l'LLM produceva immagini di stile diverso e a volte codice che falliva.	Un'unica funzione **chart()** che valida i dati e produce sia una specifica JSON (grafico React interattivo, con vista a tabella) sia il PNG con la palette dell'app.

Servizi monitorati, deploy e fonti precise

PROBLEMA	COME L'HO CAPITO	SOLUZIONE
6 **Servizi che non partono senza avvisare**	Un data agent rimasto acceso da un avvio precedente occupava la porta: il nuovo non partiva e, alla chiusura del vecchio, il caricamento di Excel/CSV falliva.	Endpoint **/api/health** che controlla tutti i servizi, porte documentate nel README e istruzioni per liberarle.
7 **Deploy online annullato**	Render concede 5 secondi al controllo di salute, ma /api/health aspettava ChromaDB e il data agent, impegnati a indicizzare 45 documenti con 0,1 di CPU.	**/api/health/live** risponde senza interrogare i servizi: il deploy è passato da 15 minuti di timeout a 44 secondi.
8 **Fonti precise**	Un chunking a lunghezza fissa avrebbe spezzato le sezioni a metà e non avrebbe permesso di citare la pagina.	Chunking per sezioni Markdown e per pagina dei PDF, con titolo e sezione dentro il testo indicizzato: il link della fonte apre il documento alla pagina citata.

ROBUSTEZZA

Dalla revisione del codice: 8 correzioni, ognuna con il suo test

Problema	Soluzione
Grafici accumulati senza limite	Retention: PNG eliminati dopo 30 giorni
Stessa analisi ripetuta da capo	Cache con chiave domanda + contesto + dataset + versione dei dati
Limite di upload solo dichiarato	Lettura a blocchi interrotta appena si supera la soglia (413)
Conversioni automatiche azzardate	Nei casi ambigui il valore resta testo, con il motivo
Due richieste sulla stessa chat si cancellavano	Lock per conversazione
Richiesta annullata, ma l'agente continuava	AbortController fino a OpenAI e al data agent, più timeout espliciti
Nessun test automatico prima del deploy	GitHub Actions + deploy solo con i test verdi
Versioni delle dipendenze non bloccate	Lock file con uv

Prestazioni e affidabilità

Misurato, non stimato

Cache delle risposte

La stessa domanda in una nuova conversazione torna subito, senza chiamare l'AI. Si invalida se cambiano documenti, dataset o modello; scade dopo 24 ore.

Risposta dalla cache

Validazione all'avvio

Il data agent controlla le 40 colonne del CSV e i tipi dopo la pulizia. Se manca una colonna non parte e dice quale; gli altri problemi compaiono in **/api/health** e nella pagina Dati.

Benchmark

Righe	Analisi più lenta	DuckDB	Memoria all'avvio
1.578	0,004 s	0,003 s	4 MB
100.000	0,04 s	0,01 s	222 MB
250.000	0,11 s	0,02 s	554 MB

Il limite è la memoria, non il tempo: pandas basta fino a circa 100.000 righe, oltre conviene DuckDB.

RISULTATI, LIMITI E PROSSIMI PASSI

Cosa funziona, cosa manca, cosa viene dopo

Risultati

- Tutti i requisiti della traccia coperti
- 3 tipi di domanda (documenti, dati, ibride) da un solo punto di accesso
- Ogni risposta verificabile
- 244 test automatici (106 backend, 138 data agent) che girano a ogni push
- Dipendenze senza vulnerabilità note
- Online su Render e Netlify, con HTTPS

Limiti onesti

- Deploy su Render con HTTPS e cookie Secure; mancano ancora header come CSP e HSTS
- Demo online sul piano gratuito: i dati caricati si azzerano a ogni riavvio (i documenti ufficiali si ricaricano da soli)
- Sandbox applicativa, non a livello di sistema operativo
- I testi dei documenti passano da OpenAI

Prossimi passi

- Permessi per categoria di documento (i metadati sono già pronti)
- Data agent in un container isolato
- Modello in locale per i dati sensibili
- Collegamento diretto ai sistemi GPS del club

Grazie

REPOSITORY GITHUB

github.com/andreagervasioDev/progetto-finale

DEMO ONLINE

progetto-management-ai.netlify.app

management-ai-r5nl.onrender.com

Dopo 15 minuti senza visite, la prima apertura impiega circa un minuto.

Codice completo (frontend, backend, data_agent), dati, knowledge base, README con installazione e avvio.

